

Due to client confidentiality, the production repository and dataset are private. This case study includes anonymized architecture, sample outputs, workflow screenshots, and a sanitized technical summary of the delivered system.

Sanitized case study repository <https://github.com/mlordjames/legal-statute-platform-case-study>

CONFIDENTIAL LEGAL TECH CLIENT · CASE STUDY

Multi-State Legal Data Platform

Founder & Data Lead, 7 Seer

Turning scattered public legal sources into structured, searchable, analytics-ready datasets at scale.

53K+

Statute sections

530K+

Court case records

4 states

TX / FL / NY / CA

12

Practice areas

Project Summary

The system transformed scattered public legal sources into structured, searchable, analytics-ready legal datasets. Statutes from multiple states and hundreds of thousands of court cases were extracted, normalized, validated, AI-enriched, and stored in a lakehouse-style architecture ready for SQL analytics and product features.

Business Problem

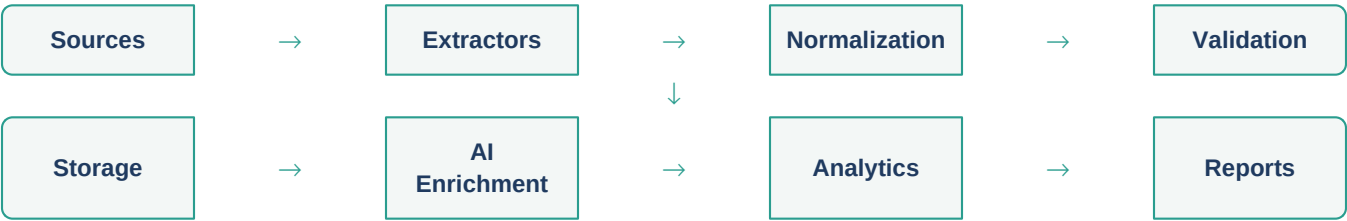
Public legal data is scattered across many state sources, published in inconsistent structures, and slow and expensive to process by hand. Building a clean, search-ready dataset from this raw material — reliably and at scale — was the core challenge.

Solution Delivered

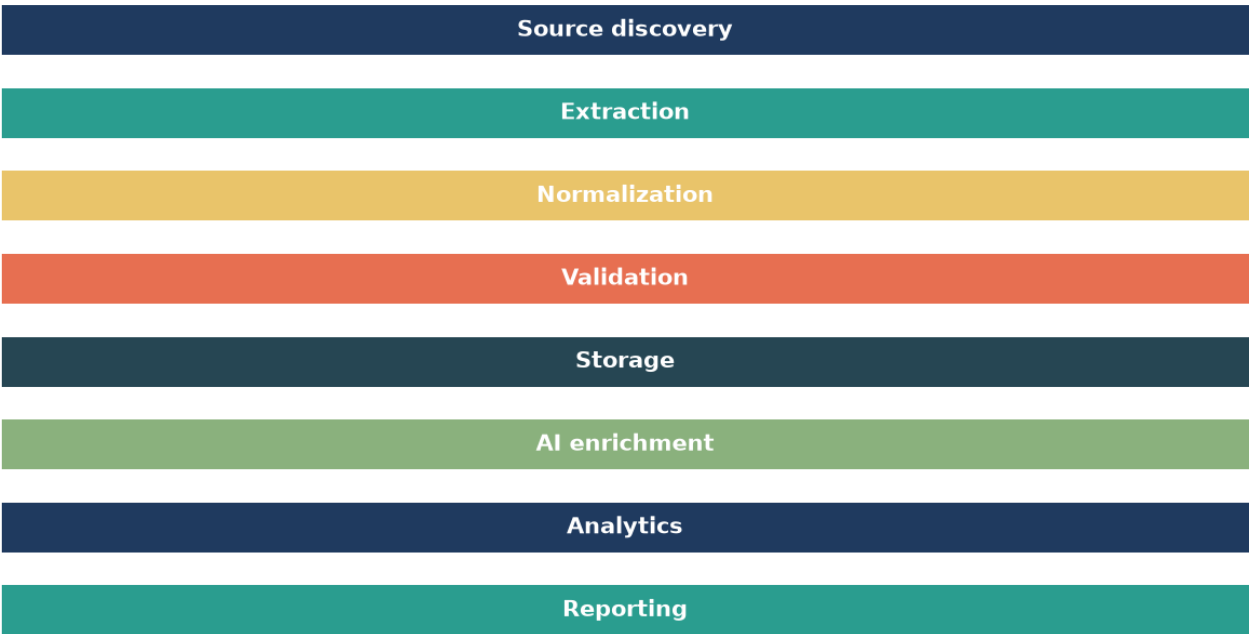
- Multi-state statute extraction
- Normalized data schemas
- AI-generated summaries and metadata
- CourtListener bulk data processing
- AWS lakehouse-style storage
- Parquet / Athena-style analytics
- Airflow-style orchestration
- Validation and checkpoint / resume workflows
- Client-ready reporting

Technical Architecture

Extraction is decoupled from enrichment, with a validated record in between. Data flows through lakehouse zones (raw → clean → enriched), is stored as Parquet, and is queried via an Athena-compatible SQL layer. The whole flow is orchestrated as discrete, re-runnable tasks.



Data Pipeline Stage Overview



Eight discrete, re-runnable pipeline stages from source discovery to reporting.

Legal Statute Pipeline — System Architecture

4 states × 12 practice areas → extraction → AI enrichment → analytics & case-law

SOURCES · 53,131 statutes

California · 25,571
california.public.law

New York · 18,528
NY Senate API

Florida · 4,236
flsenate.gov

Texas · 4,796
texas.public.law

EXTRACTION per-state scrapers & API clients · flatten · threaded fetch · parse + normalize · proxy + resume · merged_pr/ for multi-PA laws

STATUTES (JSON) results/<date>/statutes/<pa>/ · ~102M chars raw_text · 12 practice areas · state · citation · source_url

ENRICHMENT (AI) batch enrich · Claude, config-driven · + plain_english_summary · + user_intent_tags · **100% coverage · 0 failures**

ANALYTICS & DATA PLATFORM

CourtListener case-law
530,141 cases · 53+Parquet+Athena
(parallel pipeline → diagram 2)

Reports
public HTML report
client case study

Data platform
coverage / cost · quality gate (0 errors)
Parquet · DuckDB SQL

RESULTS

53,131 statute records
530,141 court cases
4 states × 12 practice areas
100% AI-summary coverage
0 data-quality errors

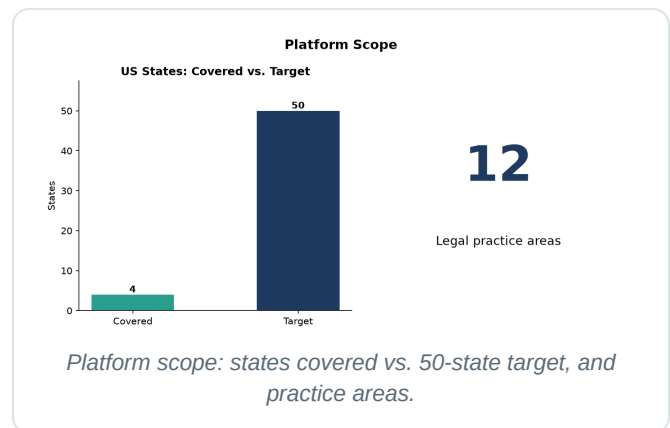
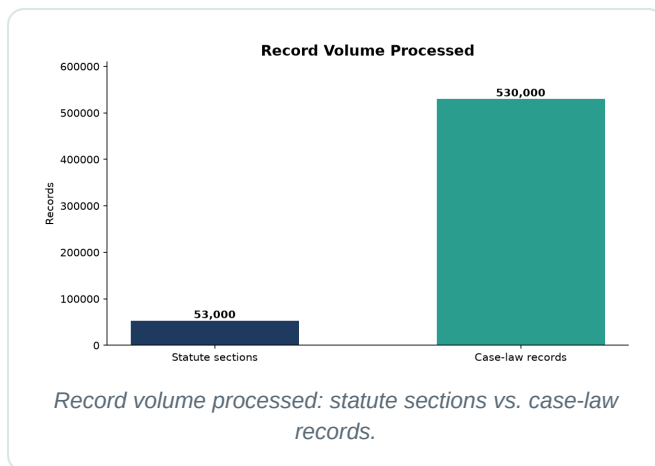
System architecture: 4 states × 12 practice areas → extraction → AI enrichment → analytics & case-law.

Tech Stack

Python	Playwright	AWS S3	Parquet	Athena
Airflow	Claude AI / LLM	CourtListener	JSON / CSV	Validation scripts

Scale and Impact

- 53,000+ statute sections processed
- 530,000+ court case records processed
- 4 US states covered (Texas, Florida, New York, California)
- 12 legal practice areas
- Designed to scale toward all **50** US states



Selected Technical Decisions

Separating extraction from enrichment so re-extracting a source never forces costly re-enrichment, and each stage can be improved independently.

Checkpoint / resume so long, interruptible state-level runs continue from the last good point instead of restarting.

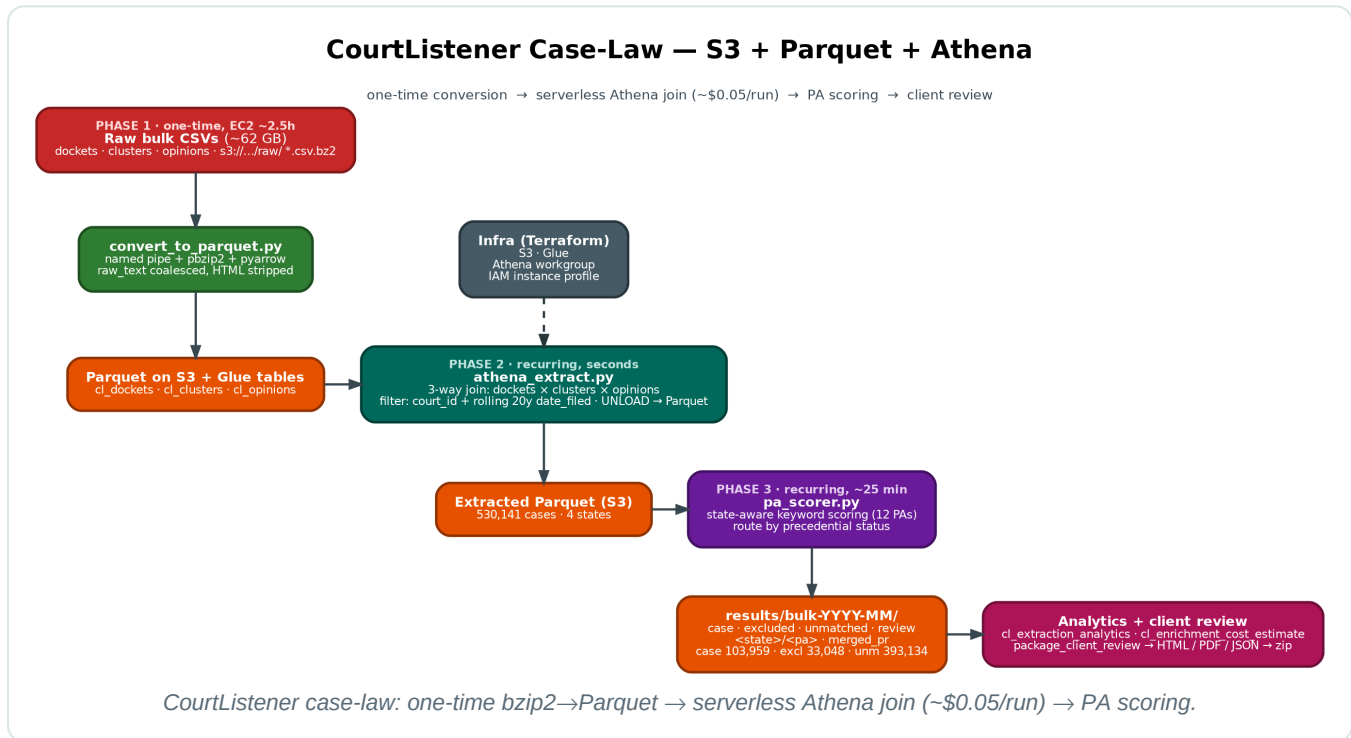
Bulk data for case law instead of slow per-record API crawling — far faster and gentler for 500k+ records.

Lakehouse-style storage (S3 + Parquet + Athena) for cheap, serverless, SQL-queryable analytics that scales with data volume.

Production repo kept private with this sanitized case study published separately to protect client confidentiality.

Engineering Deep-Dive: CourtListener at Scale

The hardest part of the platform was case law: structuring **~530,000 state court cases** from CourtListener's public corpus — a dataset whose raw opinion file alone is **54 GB compressed (~350 GB decompressed)**. Making this reliable, cheap, and scalable drove the most interesting engineering on the project.



Bottleneck	Final solution
REST API caps free use at 125 requests/day ; ~530k cases need thousands of calls.	Switch to the public bulk download (3 CSVs, ~62 GB) — months of crawling becomes a one-time copy.
54 GB opinions file is a single non-splittable bzip2 stream ; a silent decompressor crash wrote 625 of 530,034 rows and reported success.	S3 + Parquet + Athena — a managed engine owns the join, parallelism, retries, and error propagation. ~\$0.05/query.
DuckDB assumes seekable descriptors — OOM buffering COPY→S3, <i>Iseek failed</i> on a named pipe.	pyarrow open_csv on a file object — a true streaming reader, ~200 MB peak. 350 GB handled on a small box.
First convert kept <i>plain_text</i> only, but 69% of opinions are HTML-only → just ~32% case coverage.	Coalesce plain_text + 6 HTML/XML fallbacks into one cleaned raw_text — 99.98% non-empty over 10.75M opinions.
Opinions Parquet (~40–50 GB) overflowed the root disk mid-run.	Per-file stream-upload to staging + delete local immediately — ~512 MB disk peak on any volume.

Selected code

■ SAMPLE — SANITIZED | fail loudly

```
# stderr discarded + exit code unchecked =  
# a silent crash wrote 625 of 530,034 rows  
# but still reported "OK".  
proc = Popen(cmd, stdout=PIPE,  
             stderr=DEVNULL)  
for row in DictReader(proc.stdout):  
    ... # ends early, no error  
  
# FIX: gate on exit code AND on coverage.  
if proc.returncode != 0:  
    raise RuntimeError("decompress failed")  
if rows_written < expected * 0.8:  
    raise RuntimeError("too few rows")
```

■ SAMPLE — SANITIZED | pyarrow streaming

```
# bzip2 → named pipe; pyarrow streams 64MB  
# blocks, ~200MB peak. file object (not a  
# path) avoids the lseek call.  
with open(pipe_path, "rb") as pipe:  
    reader = pac.open_csv(  
        pipe,  
        ReadOptions(block_size=64 << 20),  
        ParseOptions(newlines_in_values=True),  
        ConvertOptions(include_columns=COLS))  
    for batch in reader:  
        writer.write_batch(batch) # flush now
```

■ SAMPLE — SANITIZED | recurring Athena join — ~\$0.05/run

```
UNLOAD (  
    SELECT o.cluster_id,  
        array_join(array_agg(o.raw_text), chr(10)) AS raw_text,  
        oc.case_name, oc.date_filed, d.court_id  
    FROM cl_opinions o  
    JOIN cl_clusters oc ON o.cluster_id = oc.id  
    JOIN cl_dockets d ON oc.docket_id = d.id  
    WHERE d.court_id IN ( :approved_courts ) -- from config  
        AND oc.date_filed >= DATE '2006-01-01'  
        AND length(trim(o.raw_text)) > 0  
    GROUP BY o.cluster_id, oc.case_name, oc.date_filed, d.court_id  
) TO 's3://<data-lake>/athena-results/run=<ts>/'  
WITH (format='PARQUET', compression='SNAPPY')
```

Cost Engineering

Ephemeral, auto-terminating compute conversion runs on a short-lived EC2 box with a -- shutdown flag (sudo shutdown -h now) that self-terminates on success, never on failure — no idle instances.

Disk-bounded streaming + cleanup each ~512 MB Parquet file is uploaded then deleted immediately, so a multi-hundred-GB job needs ~512 MB of disk; staging→verify→promote keeps the canonical dataset clean.

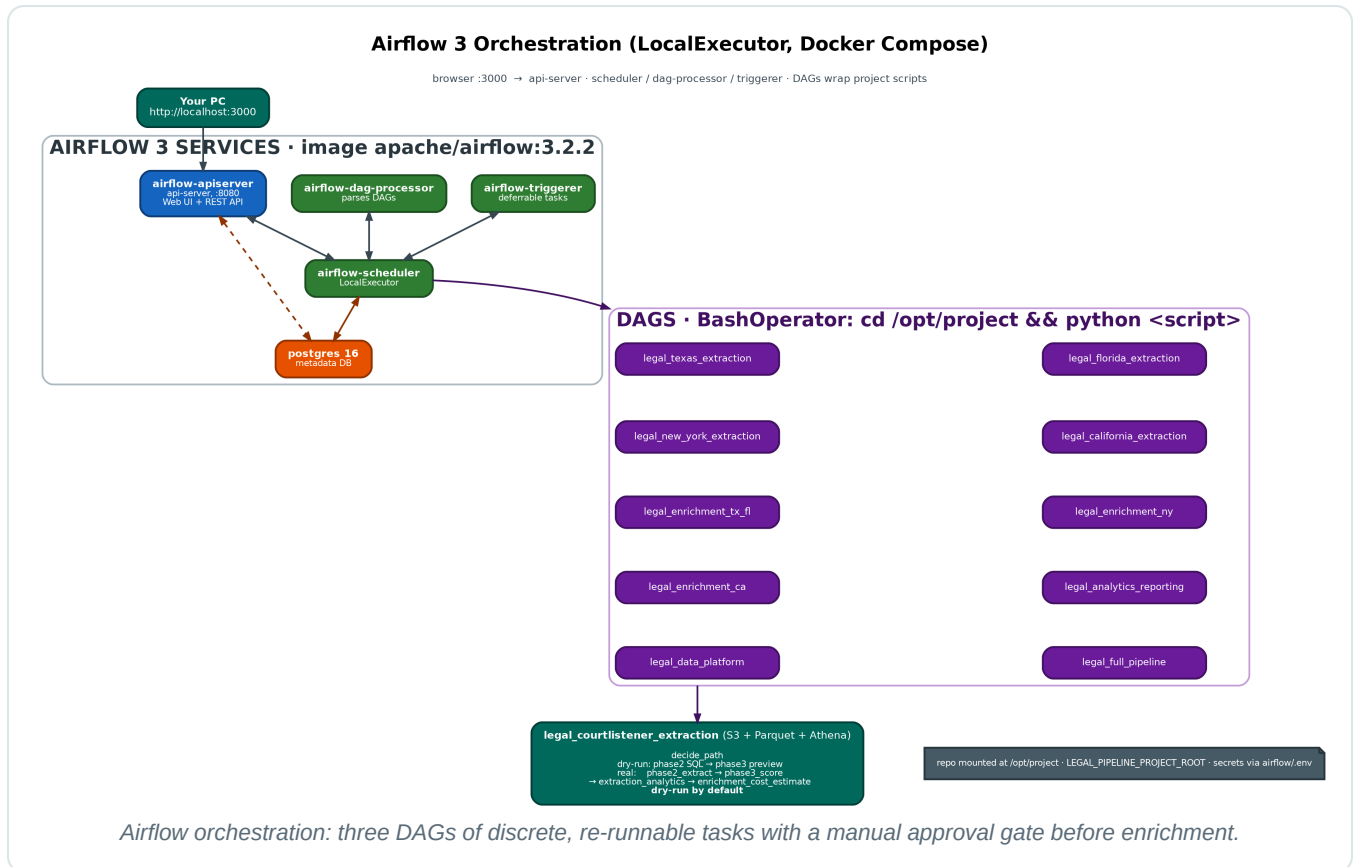
Convert once, query forever bzip2→Parquet is one-time per monthly snapshot; every query reads the same column-pruned Parquet (~\$0.05) instead of re-downloading 62 GB.

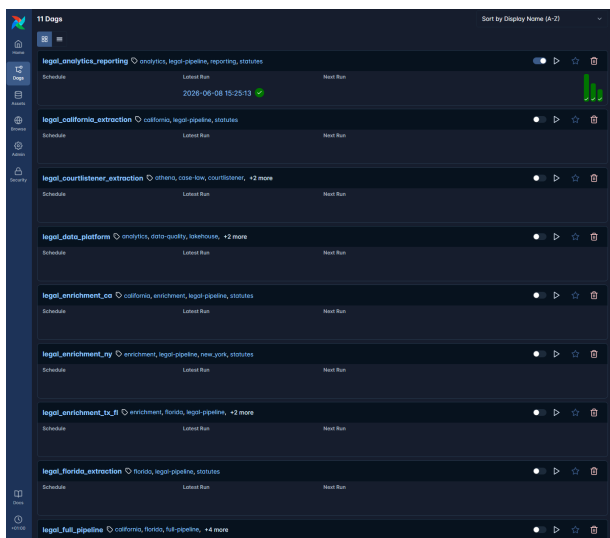
Scale-flat querying + planned batch enrichment adding states is a config change (Athena stays ~\$0.05/run for 4 states or 50); case-law LLM enrichment routes through the Batch API with model right-sizing to ~halve spend.

All-in recurring cost: ~\$0.40 per run (Athena ~\$0.05 + PA scoring ~\$0.09 + S3 writes ~\$0.27); one-time Parquet conversion ~\$0.43 + ~\$1/month storage.

Orchestration & Workflow

The pipeline runs as three Airflow DAGs, with each stage modeled as a discrete, re-runnable task so a failure resumes from the last good point. AI enrichment is **dry-run by default** and gated behind a manual approval checkpoint before any paid LLM calls are made.





Screenshot from the running Airflow UI: the DAGs list with the three pipeline DAGs and their schedules.



Screenshot from the running Airflow UI: the legal_full_pipeline run graph showing per-stage task dependencies.

Sample Output Preview

Synthetic, sanitized records only — the full raw_text field is excluded.

SAMPLE — SANITIZED | statute record

```
{
  "state": "texas",
  "practice_area": "landlord_tenant",
  "chapter": "Property Code Chapter 92",
  "section_id": "sample_section_001",
  "title": "Sample Residential Tenancy",
  "extraction_status": "validated"
}
```

SAMPLE — SANITIZED | enrichment output

```
{
  "section_id": "sample_section_001",
  "summary": "Plain-English summary for search and user understanding.",
  "intent_tags": ["tenant_rights", "lease_compliance", "notice_requirements"],
  "complexity_level": "medium",
  "validation_status": "passed"
}
```

Confidentiality Note

Production code, datasets, client details, credentials, raw legal text, and private prompts are excluded. This document and its repository contain only sanitized architecture, synthetic samples, diagrams, and summaries.

About 7 Seer

7 Seer builds data pipelines, automation systems, AI-enriched datasets, and cloud data platforms for businesses that need reliable data collection, processing, and analytics.

Project Credits

Project Developer	Etiese James — Lead Data Engineer, 7 Seer
Lead Agent	Claude Code (Opus 4.8)
Reviewer Agent	OpenAI GPT